Anomaly Detection in Credit Card Transactions

1. Abstract

This project, "Anomaly Detection in Credit Card Transactions", aims to develop an intelligent system that can identify unusual patterns and anomalies in credit card transactions. The system utilizes machine learning algorithms to analyse transactional data and detect potential fraudulent activities, allowing for timely intervention and minimizing financial losses. By leveraging advanced data analytics and anomaly detection techniques, this project seeks to improve the security and reliability of credit card transactions, ultimately enhancing the overall customer experience. The system's effectiveness is evaluated through a comprehensive analysis of its accuracy, precision, and recall in detecting anomalies, providing valuable insights for further improvement.

2. Domain Introduction

The domain of credit card transactions is a complex and high-stakes environment, with billions of transactions taking place every day. The widespread use of credit cards has led to an increase in fraudulent activities, resulting in significant financial losses for individuals, businesses, and financial institutions. According to recent statistics, credit card fraud accounts for a substantial percentage of overall payment card fraud, with most cases involving unauthorized transactions, identity theft, and card skimming.

The credit card industry has implemented various security measures to mitigate these risks, including the use of chip technology, encryption, and two-factor authentication. However, these measures are not foolproof, and new threats continue to emerge. Anomaly detection in credit card transactions has become a critical component of fraud prevention strategies, enabling the identification of unusual patterns and suspicious activities that may indicate potential fraud. By leveraging advanced data analytics and machine learning techniques, anomaly detection systems can help reduce the risk of credit card fraud, protecting both consumers and financial institutions from financial losses.

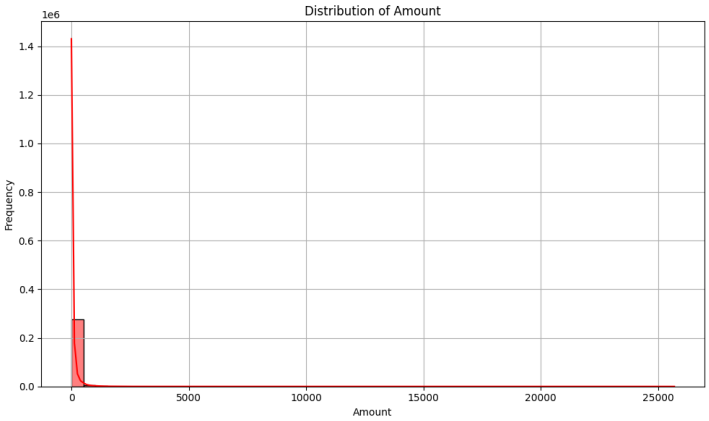
3. Dataset Description

This dataset contains anonymized credit card transaction data for fraud detection. The dataset includes 31 columns, with features extracted using Principal Component Analysis (PCA) to protect sensitive financial information. The Time column represents the elapsed time since the first transaction, and the Amount column represents the transaction value. The Class column indicates whether a transaction is fraudulent (1) or legitimate (0).

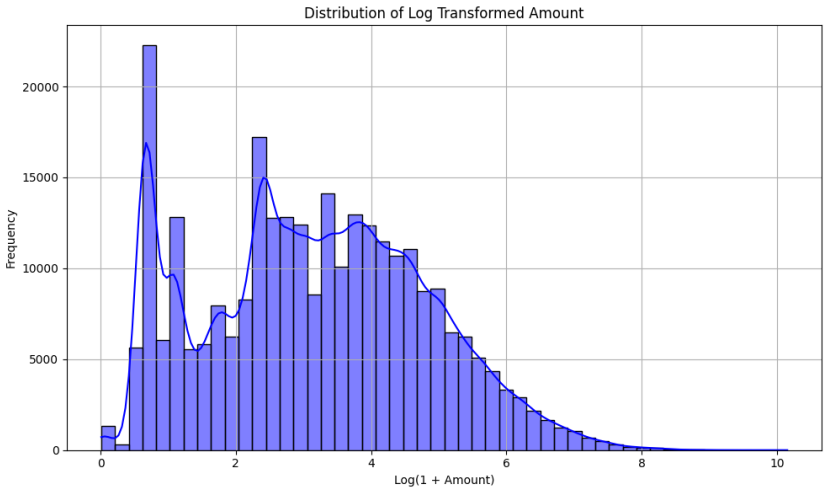
4. Implementation methodology with brief justification

4.1 Data Preprocessing

The distribution of transaction amounts showed significant skewness due to a high frequency of lower values.



To address this, transactions with non-positive amounts were filtered out. A log transformation (np.log1p()) was then applied to normalize the data, reducing skewness and making the distribution more symmetric for better analysis and modelling.



For feature engineering, the 'Time' column was converted to an 'Hour' feature by extracting the hour of the day. To capture its cyclical nature, 'Hour_sin' and 'Hour_cos' features were created using sine and cosine transformations. The original 'Time' and 'Hour' columns were then dropped for further modelling.

4.2 LSTM AutoEncoder Model Architecture

The LSTM AutoEncoder model architecture is designed to effectively capture temporal dependencies in credit card transaction data, enabling the identification of anomalies. The architecture consists of two primary components: the encoder and the decoder.

Encoder

The encoder is responsible for processing input sequences and learning a compressed representation (latent space) of the transactions. This is achieved through the use of LSTM layers, which allow the model to capture long-range dependencies in the data. The encoder takes in a sequence of transactions and outputs a fixed-length vector representation, which is then passed to the bottleneck.

Bottleneck

The bottleneck is the core of the AutoEncoder, where the compressed representation of the input data is learned. In this architecture, we utilize an LSTM layer in the bottleneck to further refine the representation and capture any remaining temporal dependencies. The LSTM layer in the bottleneck enables the model to learn a more robust and compact representation of the input data.

Decoder

The decoder attempts to reconstruct the original input sequences from the compressed representation learned by the encoder and bottleneck. The decoder consists of LSTM layers that mirror the structure of the encoder, allowing the model to effectively reverse the encoding process and generate a reconstructed output.

In this project, we implemented an **LSTM AutoEncoder** model to effectively detect anomalies in credit card transaction data. The approach capitalizes on the capacity of LSTM (Long Short-Term Memory) networks to capture temporal dependencies in sequential data, making it well-suited for time-series analysis inherent in transaction records. The architecture facilitates the encoding of transaction sequences into a lower-dimensional representation, from which the decoder attempts to reconstruct the original input. Anomalies are identified based on the reconstruction error, as these represent deviations from the expected transaction patterns.

4.3 Optimizer: Adam (Adaptive Moment Estimation)

The optimization process is conducted using the Adam optimizer, configured as follows:

optimizer = optim.Adam(model.parameters(), lr=learning_rate, weight_decay=1e-5)

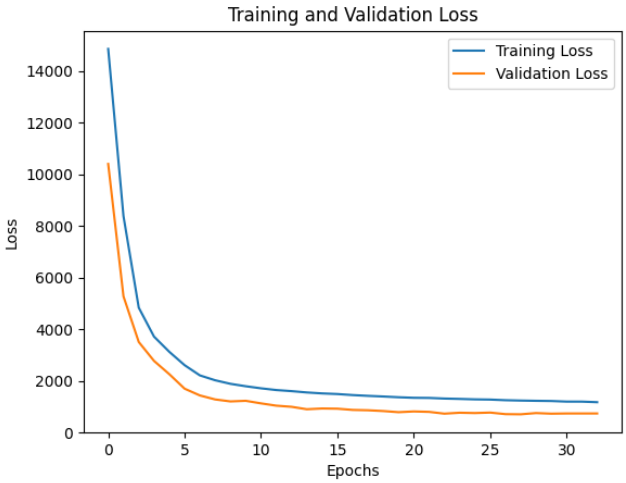
Adam is selected as the optimizer due to several advantages:

1. **Automation and Improvement in Gradient Descent:** Adam automates the process of tuning the learning rate and optimizes the gradient descent steps, allowing for more efficient convergence.

2. **Faster Convergence:** By dynamically adjusting the learning rate for each parameter based on the first and second moments of the gradients, Adam helps the model converge faster compared to traditional stochastic gradient descent.

3. **Prevention of Overfitting through Regularization:** The inclusion of L2 regularization via the weight decay term (set to 1e-5) penalizes excessively large weights, thus effectively mitigating overfitting and enhancing the model's generalization capability.

4. **Control over Learning Rate:** The specified learning rate plays a critical role in ensuring a balance between rapid convergence and stable training. Tuning the learning rate appropriately enables the model to escape local minima while maintaining learning stability.



4.4 Reconstruction Loss (MAE)

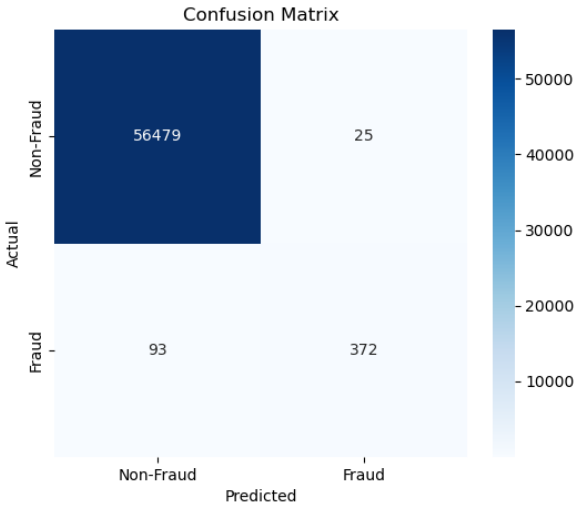
The reconstruction loss, defined as Mean Absolute Error (MAE) in this case, quantifies how effectively the AutoEncoder reconstructs the input transactions. MAE is advantageous for this application as it provides a clear metric for evaluating the performance of the model in capturing normal patterns. Transactions with high reconstruction loss values are flagged as anomalies, indicating that they significantly deviate from the established norm.

```
criterion = nn.L1Loss(reduction='sum')

def reconstruction_loss(recon_x, x):

    return criterion(recon_x, x)
```

This methodology combines the strengths of LSTM networks, appropriate loss functions, and advanced optimization techniques, making it a robust solution for detecting anomalies in credit card transactions.

5. Results and Discussion

In this section, we present and analyse the performance of our fraud detection model using multiple evaluation metrics, including the confusion matrix, classification performance metrics, and the ROC curve.

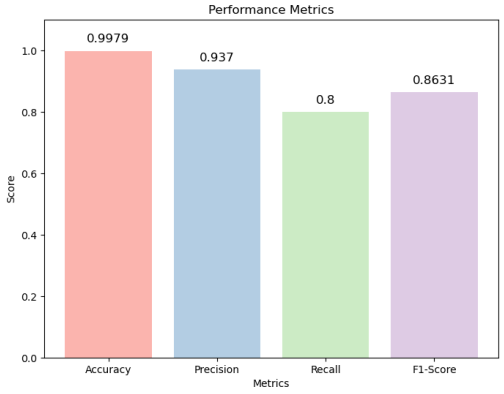


5.1 Confusion Matrix Analysis

The confusion matrix, depicted in Figure 1, provides insights into the classification performance of our model. The matrix shows:

- **True Positives (TP):** 372 fraudulent transactions correctly identified as fraud.
- **True Negatives (TN):** 56,479 non-fraudulent transactions correctly identified as non-fraud.
- **False Positives (FP):** 25 non-fraudulent transactions incorrectly classified as fraud.
- **False Negatives (FN):** 93 fraudulent transactions misclassified as non-fraud.

The model demonstrates a strong ability to distinguish between fraudulent and non-fraudulent transactions, with a minimal number of misclassifications.

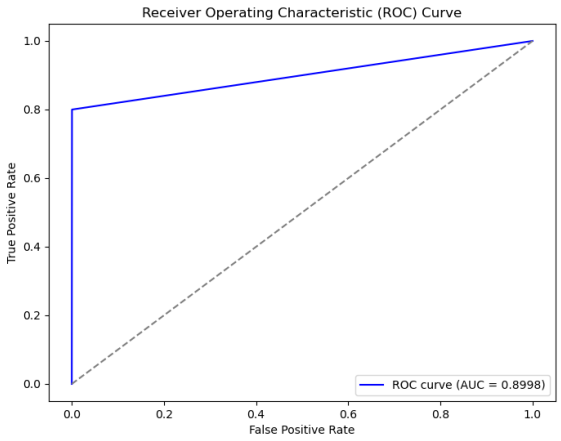


5.2 Performance Metrics

The key performance metrics of the model, as illustrated in Figure 2, include:

- **Accuracy:** 0.9979, indicating that the model correctly classifies 99.79% of transactions.
- **Precision:** 0.937, meaning that 93.7% of transactions identified as fraud are actually fraudulent.
- **Recall:** 0.8, signifying that the model detects 80% of actual fraudulent transactions.
- **F1-Score:** 0.8631, providing a balance between precision and recall.

The high accuracy and precision suggest that the model effectively minimizes false positives, reducing unnecessary fraud alerts. However, the recall value of 0.8 indicates that some fraudulent transactions are still going undetected.



5.3 ROC Curve and AUC Score

Figure 3 presents the ROC curve, which visualizes the trade-off between the true positive rate (sensitivity) and false positive rate. The Area Under the Curve (AUC) is 0.8998, signifying strong discriminatory power. An AUC close to 1 indicates that the model effectively differentiates between fraudulent and non-fraudulent transactions.

5.4 Discussion

The results indicate that the model is highly effective in identifying fraud, with strong accuracy and precision. However, the recall value suggests that some fraudulent cases are still being missed. To improve recall, techniques such as adjusting classification thresholds, employing cost-sensitive learning, or implementing ensemble methods could be explored. Additionally, incorporating more advanced feature engineering and leveraging deep learning techniques may further enhance performance.

6. Future Scope

While the current model performs well, there is still room for improvement and expansion:

- **Real-time deployment**: Integrate the model into a live system that flags high-risk transactions in real time, providing alerts or secondary checks.
- **Advanced architectures**: Explore hybrid models combining LSTM with **CNNs** or **Transformers** to capture both temporal and spatial patterns.
- **Synthetic data generation**: Use **SMOTE** or **GANs** to synthetically generate more fraud cases to help improve recall and model robustness.
- **Explainability**: Incorporate **SHAP** or **LIME** to provide interpretability on why a particular transaction was flagged as fraudulent.
- **Feedback loop**: Design the system to learn from new confirmed fraud cases over time, evolving and improving continuously.

This project lays a solid foundation for intelligent fraud detection, and with further optimization, it can be extended to industry-grade applications.